

Stacks in C++

Data Structures · CT077-2-3-DSTR · Detailed Study Notes

 LIFO / FILO Concept

 5 Core Operations

 Postfix Evaluation

 Infix → Postfix

 Array & Linked List Impl.

TABLE OF CONTENTS

- 1 What is a Stack? — LIFO & FILO Explained
- 2 Real-Life Examples of Stacks
- 3 Operation ① — Push (Add to Top)
- 4 Operation ② — Pop (Remove from Top)
- 5 Operation ③ — Peek / Top (Look at Top)
- 6 Operation ④ — Empty (Is the Stack Empty?)
- 7 Operation ⑤ — Size (How Many Elements?)
- 8 Application ① — Balancing Parentheses
- 9 Application ② — Evaluating Postfix Expressions
- 10 Application ③ — Infix to Postfix Conversion
- 11 Implementation — Using Array & Linked List
- 12 STL `stack<>` — C++ Built-In Stack
- 13 Common Mistakes & Quick Reference

1. What is a Stack?

DEFINITION

A **stack** is a linear data structure that follows a strict rule about the order in which elements are added and removed. That rule is called **LIFO** — or its mirror name **FILO**.

LIFO — Last In, First Out

The **last** element you added is the **first** one you remove.

FILO — First In, Last Out

The **first** element you added is the **last** one to come out.

Both LIFO and FILO describe the *exact same behaviour* — just said from a different angle. Your lecturer may use either term.

THE STACK OF BOOKS ANALOGY

Imagine a stack of books on a desk:

- You add books **one at a time on top**
 - You can only take the **top book off first**
 - You cannot pull Book A out from the bottom without removing D, C, B first
 - Only the **top element is ever accessible**
- ← **TOP** (last added)
- 📖 **Book D**

📖 **Book C**

📖 **Book B**

📖 **Book A**
- ↑ **BOTTOM** (first added)
- 💡 This is exactly how a stack data structure works. The top of the stack is the only place you can add or remove elements.

KEY TERMINOLOGY

Term	Meaning
Top	The current highest element — the only one you can access
Push	Adding a new element onto the top of the stack
Pop	Removing the top element from the stack
Peek	Looking at the top element <i>without</i> removing it
Stack Overflow	Trying to push onto a full stack (array implementation)
Stack Underflow	Trying to pop from an empty stack

2. Real-Life Examples of Stacks

① Reversing a String — "Hello" → "olleH"

Push each character onto the stack: H, e, l, l, o. Then pop them off one by one — the last character pushed (o) comes out first, giving you "olleH". This is LIFO in action.

② Call History on Your Phone

Each new call is pushed onto the history stack. The most recent call always appears at the top. Oldest calls are buried at the bottom.

③ Web Browser Back Button

Every page you visit is pushed onto a stack. When you click Back, the current page is popped off and you land on the previous one. Facebook → YouTube → Instagram → [Back] → YouTube.

④ Undo in Text Editors (Ctrl+Z)

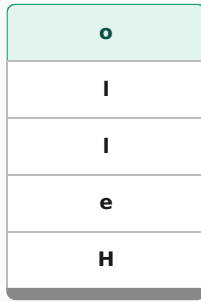
Every action you perform is pushed onto an undo stack. Pressing Ctrl+Z pops the most recent action and reverses it.

⑤ Function Call Stack in Programming

When your program calls a function, it's pushed onto the call stack. When the function returns, it's popped off. This is why recursive functions can cause "stack overflow" — too many pushes!

REVERSING "HELLO" — STEP BY STEP VISUAL

PUSH each char



"Hello" pushed

POP each char



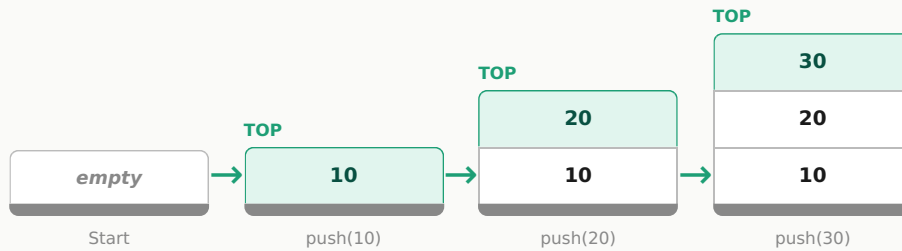
o → l → l → e → H

① Push — Adding an Element to the Stack O(1)

What does Push do?

Push adds a new element to the **top** of the stack. It is always $O(1)$ — constant time — because we always add to the same place (the top). No traversal needed.

VISUAL — PUSHING 10, 20, 30 ONE AT A TIME



C++ CODE — PUSH USING STL STACK

```
#include <stack>
using namespace std;

stack<int> s;

s.push(10); // Stack: [10]           ← top
s.push(20); // Stack: [10, 20]      ← top
s.push(30); // Stack: [10, 20, 30] ← top
```

② Pop — Removing the Top Element O(1)

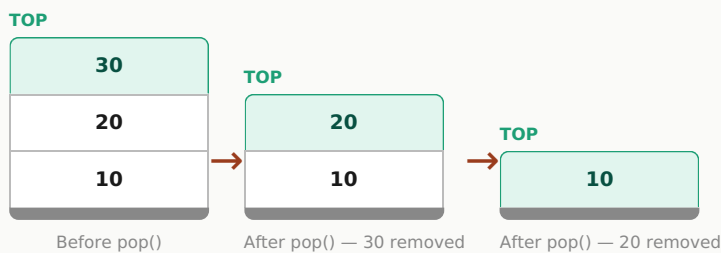
What does Pop do?

Pop **removes the top element** from the stack. In C++ STL, `pop()` removes it but does NOT return its value — you must use `top()` first if you want to read the value before removing.

🚨 Stack Underflow!

Never call `pop()` on an empty stack — it causes undefined behaviour (a crash). Always check `empty()` first.

VISUAL — POPPING FROM THE STACK



C++ CODE — SAFE POP PATTERN

```
// WRONG — crashes if stack is empty!
s.pop();

// CORRECT — always check empty() first
if (!s.empty()) {
    cout << "Removing: " << s.top() << endl; // Read first
    s.pop();                                // Then remove
}
```

③ Peek / Top — Look at the Top Without Removing O(1)

What does Peek do?

Peek (also called **Top**) lets you *read* the top element without removing it. Think of it like looking at the top book in a pile without picking it up.

TOP ← peek() reads THIS

30 👁
20
10

Stack unchanged after peek()

```
// In STL it's called top()
int val = s.top(); // val = 30
// Stack still has 30, 20, 10
```

④ Empty — Is the Stack Empty? O(1)

What does Empty do?

Returns `true` if the stack has no elements, `false` otherwise. You should check this **before every Pop or Peek** to prevent crashes.

no elements	5
	3

empty() → true

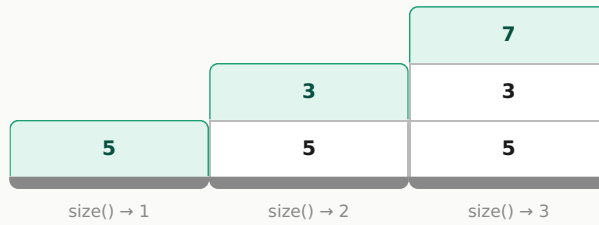
empty() → false

```
if (s.empty()) {
    cout << "Stack is empty!" << endl;
} else {
    cout << "Top is: " << s.top() << endl;
}
```

⑤ Size — How Many Elements Are in the Stack? O(1)

What does Size do?

Returns the **number of elements** currently in the stack as an integer.



```
cout << "Stack has " << s.size() << " elements" << endl;
```



8. Application ① — Balancing Parentheses

The Problem

Given a string of brackets — `( ) [ ] { }` — check if every opening bracket has a matching closing bracket in the correct order. This is used by compilers and code editors to detect syntax errors.

Valid:

`( [ ] ) { }` ✓ Balanced

Invalid:

`( [ )` ✗ Wrong closing bracket

`( ( )` ✗ Missing closing bracket

ALGORITHM — HOW TO SOLVE IT WITH A STACK

- 1 Scan the string from **left to right**, character by character.
- 2 If you see an **opening bracket** `( [ {` → **push it** onto the stack.
- 3 If you see a **closing bracket** `) ] }`:
 - If the stack is empty → **INVALID** (no matching opener)
 - If top of stack is the matching opener → **pop it**
 - If top doesn't match → **INVALID**
- 4 At the end: if stack is **empty** → **VALID**. If still has items → **INVALID** (unclosed brackets).

C++ CODE

```
#include <stack>
#include <string>
using namespace std;

bool isBalanced(string expr) {
    stack<char> s;

    for (char c : expr) {
        // Push all opening brackets
        if (c == '(' || c == '[' || c == '{') {
            s.push(c);
        }
        // For closing brackets, check against top
        else if (c == ')' || c == ']' || c == '}') {
            if (s.empty()) return false; // Nothing to match
            char top = s.top(); s.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{'))
                return false; // Mismatch!
        }
    }
    return s.empty(); // Valid only if nothing left
}

int main() {
    cout << isBalanced("([{}])") << endl; // 1 (true)
    cout << isBalanced("([)]") << endl; // 0 (false)
}
```



9. Application ② — Evaluating Postfix Expressions

Infix vs Postfix — What's the Difference?

Type	Format	Example	Used by
Infix	Operator <i>between</i> operands	5 + 3	Humans (normal maths)
Postfix	Operator <i>after</i> operands	5 3 +	Computers (easier to parse)
Prefix	Operator <i>before</i> operands	+ 5 3	Less common

! Why does a computer prefer Postfix?

Postfix needs no parentheses and no rules about operator precedence (BODMAS). A computer can evaluate it in a single left-to-right scan using a stack.

ALGORITHM — 3 SIMPLE RULES

- 1 Scan the postfix expression **from left to right**.
- 2 If the token is a **number** → **push it** onto the stack.
- 3 If the token is an **operator (+, -, *, /)**:
 - Pop the **top two numbers** off the stack
 - Apply the operator: `result = second_popped OP first_popped`
 - Push the result back onto the stack
- 4 At the end, the **one remaining value** in the stack is the answer.

FULL WORKED EXAMPLE — FROM THE SLIDES

Evaluate: 5, 6, 2, +, *, 12, 4, /, -

This is the postfix version of: `((5 * (6 + 2)) - (12 / 4))`

Token	Action	Stack (bottom → top)
5	Push 5	[5]
6	Push 6	[5, 6]
2	Push 2	[5, 6, 2]
+	Pop 2 & 6 → 6+2=8 → Push 8	[5, 8]
*	Pop 8 & 5 → 5×8=40 → Push 40	[40]
12	Push 12	[40, 12]
4	Push 4	[40, 12, 4]
/	Pop 4 & 12 → 12/4=3 → Push 3	[40, 3]
-	Pop 3 & 40 → 40-3=37 → Push 37	[37]
Final Answer		37 ✓

⚠ Order of operands matters!

When you pop two values for subtraction or division, the *second* popped value goes on the LEFT. So for "-": pop 3 (first), pop 40 (second) → compute `40 - 3 = 37`, NOT `3 - 40`.

C++ CODE — POSTFIX EVALUATOR

```
#include <stack>
#include <sstream>
using namespace std;

int evalPostfix(string expr) {
    stack<int> s;
    stringstream ss(expr);
    string token;

    while (ss >> token) {
        if (isdigit(token[0])) {          // It's a number
            s.push(stoi(token));
        } else {                          // It's an operator
            int b = s.top(); s.pop(); // First popped = right
            int a = s.top(); s.pop(); // Second popped = left
            if (token == "+") s.push(a + b);
            else if (token == "-") s.push(a - b);
            else if (token == "*") s.push(a * b);
            else if (token == "/") s.push(a / b);
        }
    }
    return s.top(); // Final answer
}

int main() {
    cout << evalPostfix("5 6 2 + * 12 4 / -"); // Output: 37
}
```

🔧 10. Application ③ — Infix to Postfix Conversion

Why Convert?

We write maths in **infix** (`a + b * c`) because it's readable for humans. Computers prefer **postfix** because there are no brackets or precedence rules to figure out — just scan left to right with a stack.

The conversion itself also uses a stack — this time to hold **operators**, while operands (letters/numbers) go directly to the output.

OPERATOR PRECEDENCE RULES

Operator	Precedence	Meaning
* /	Higher (2)	Multiplication and division evaluated first
+ -	Lower (1)	Addition and subtraction evaluated after
()	Special	Always push (; pop until (when you see)

ALGORITHM — STEP BY STEP RULES

- 1 Scan the infix expression **left to right**.
- 2 If the token is an **operand** (letter or number) → **add it directly to the postfix output**.
- 3 If the token is (→ **push it** onto the operator stack.
- 4 If the token is an **operator** (+, -, *, /):
 - While stack is NOT empty AND top of stack has **equal or higher precedence** → pop and add to output
 - Then **push the current operator**
- 5 If the token is) → pop operators to output **until you find** the matching (. Discard both brackets.
- 6 At the end → **pop all remaining operators** from the stack to the output.

WORKED EXAMPLE — FROM THE SLIDES: A + B * C + D

Expected postfix output: a b c * + d +

Token	Action	Operator Stack	Postfix Output
a	Operand → output	[]	a
+	Push +	[+]	a
b	Operand → output	[+]	a b
*	* > + (higher precedence) → push *	[+, *]	a b
c	Operand → output	[+, *]	a b c
+	Pop * (higher), pop + (equal) → output both; push new +	[+]	a b c * +
d	Operand → output	[+]	a b c * + d
(end)	Pop remaining + → output	[]	a b c * + d +
Final Postfix			a b c * + d +

ANOTHER EXAMPLE — ((5 * (6 + 2)) - (12 / 4))

This is the infix form of the postfix expression 5 6 2 + * 12 4 / - that we evaluated in Section 9.

INFIX (Maths Brain)

((5 * (6 + 2)) - (12 / 4))

How humans write it — needs BODMAS rules

POSTFIX (Computer Brain)

5 6 2 + * 12 4 / -

How computers prefer it — no brackets needed

11. Implementation — Array vs Linked List

A stack is an **abstract data type** — it defines *what* the operations are, not *how* they are stored. You can implement it in two ways:

Implementation 1 — Using an Array

Use an array to hold elements and an integer `top` to track the index of the current top element. Start `top = -1` (empty).

Pro	Con
✓ Simple and fast	✗ Fixed maximum size
✓ Cache-friendly memory	✗ Wastes memory if mostly empty
✓ No pointer overhead	✗ Stack overflow if full

```

const int MAX = 100;
int arr[MAX];
int top = -1; // -1 means empty

// Push
void push(int val) {
    if (top == MAX - 1) { cout << "Stack Overflow!"; return; }
    arr[++top] = val;
}

// Pop
int pop() {
    if (top == -1) { cout << "Stack Underflow!"; return -1; }
    return arr[top--];
}

// Peek
int peek() { return (top == -1) ? -1 : arr[top]; }
bool isEmpty() { return top == -1; }
int size() { return top + 1; }

```

Implementation 2 — Using a Linked List

The **head of the linked list acts as the top of the stack**. Push = insert at head ($O(1)$). Pop = delete from head ($O(1)$). The list grows and shrinks dynamically — no size limit.

Pro	Con
✓ Dynamic size — grows as needed	✗ Extra memory for pointers
✓ No stack overflow risk	✗ Slightly slower (heap allocation)

```

struct Node {
    int data;
    Node* next;
};
Node* top = nullptr; // Head = top of stack

// Push = Insert at head
void push(int val) {
    Node* newNode = new Node{val, top};
    top = newNode;
}

// Pop = Delete from head
int pop() {
    if (!top) { cout << "Underflow!"; return -1; }
    int val = top->data;
    Node* tmp = top;
    top = top->next;
    delete tmp;
    return val;
}

int peek() { return top ? top->data : -1; }
bool isEmpty() { return top == nullptr; }

```

💡 12. C++ STL stack<> — The Built-In Stack

C++ comes with a ready-made stack in the STL. Use `#include <stack>`. It uses a `deque` internally by default, but behaves as a pure LIFO stack.

Method	What it does	Speed
<code>push(val)</code>	Add element to top	O(1)
<code>pop()</code>	Remove top element (no return value!)	O(1)
<code>top()</code>	Return top element (no removal)	O(1)
<code>empty()</code>	Returns true if no elements	O(1)
<code>size()</code>	Returns number of elements	O(1)

 STL pop() does NOT return a value!

This is a common mistake. You must call `top()` first to get the value, then `pop()` to remove it. This is by design (exception safety).

COMPLETE EXAMPLE — MUSIC PLAYLIST STACK

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    stack<string> playlist;

    // Push songs onto the stack
    playlist.push("Taylor Swift - Shake It Off");
    playlist.push("Adele - Hello");
    playlist.push("Doja Cat - Woman");

    // Peek — who's on top?
    cout << "Now playing: " << playlist.top() << endl;
    // Output: Doja Cat - Woman

    // Pop — remove top song
    playlist.pop();
    cout << "Now playing: " << playlist.top() << endl;
    // Output: Adele - Hello

    cout << "Songs left: " << playlist.size() << endl;
    // Output: 2

    // Pop all songs out (LIFO order)
    while (!playlist.empty()) {
        cout << playlist.top() << endl;
        playlist.pop();
    }
    // Output: Adele - Hello
    //           Taylor Swift - Shake It Off
}
```

13. Common Mistakes & Quick Reference

TOP MISTAKES BEGINNERS MAKE

- ✗ **Calling pop() on an empty stack** — always call `empty()` first. A pop on an empty stack is undefined behaviour and will crash your program.
- ✗ **Forgetting that STL pop() has no return value** — read with `top()` THEN remove with `pop()`. Many students write `int x = s.pop()` — this won't compile!
- ✗ **Wrong operand order in postfix evaluation** — when you pop two values, the second popped is the LEFT operand. `pop b, pop a` → compute `a OP b`.
- ✗ **Forgetting to delete nodes in linked list implementation** — when you pop a node, `delete` it or you'll get a memory leak.
- ✗ **Infix to Postfix: not popping equal-precedence operators** — when you see a + and the stack top is also +, you MUST pop the existing + before pushing the new one. Same level = pop first.

OPERATIONS — COMPLEXITY CHEAT SHEET

Push

Add element to top.

O(1) — Constant

Pop

Remove top element.

O(1) — Constant

Peek / Top

Read top without removing.

O(1) — Constant

Empty

Check if no elements.

O(1) — Constant

Size

Count of elements.

O(1) — Constant

Search

No direct search — pop until found.

O(n) — Linear

🔑 The Big Takeaway

Every core stack operation (push, pop, peek, empty, size) is O(1) — instant. This makes stacks extremely efficient. The trade-off is that you can **ONLY** access the top element — no random access, no searching without destroying the stack.

STACK VS OTHER DATA STRUCTURES

Feature	Stack	Array	Linked List
Access	Top only	Any index $O(1)$	Walk from head $O(n)$
Insert	Top only $O(1)$	$O(n)$ (shift)	$O(1)$ at head
Delete	Top only $O(1)$	$O(n)$ (shift)	$O(n)$ to find
Order rule	LIFO	None	None
Best use	Undo, backtrack, parse	Random access	Dynamic insert/delete

WHAT COMES NEXT

✓ Done — Stacks

Push, Pop, Peek, Empty,
Size, and 3 applications

➤ Next — Queue

FIFO (First In, First Out):
Enqueue, Dequeue, Front,
Rear

📅 Later — Tree

Binary trees, traversal
(preorder, inorder,
postorder)